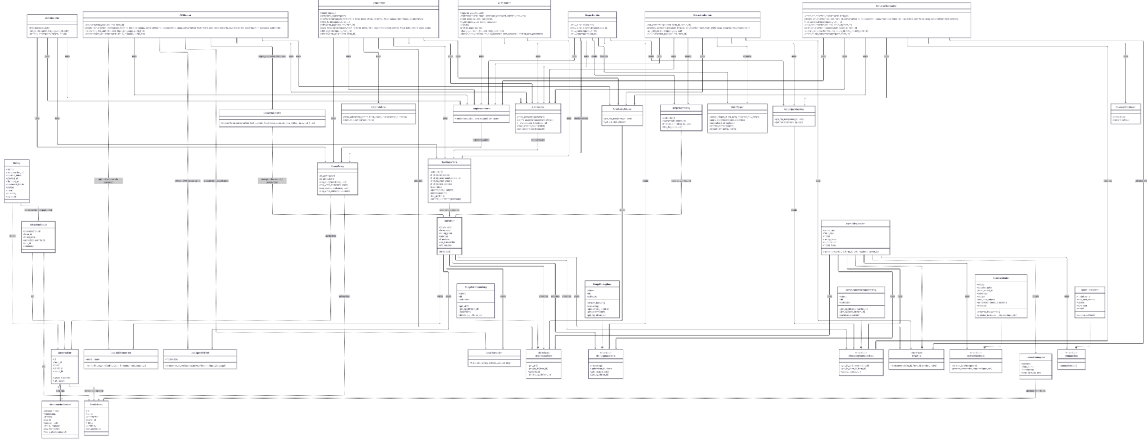


Product Design

Design Model



Link to drive with the Class Diagram in better quality: https://drive.google.com/drive/folders/1_8pBpr3-8CVtVWu3x0C0y0mhUAtof1uy?usp=sharing

This drive also consists of all the sequence diagrams for reference and clarity.

Consider the summary as follows:

Conversation	Class state • Stores information about conversations including conversation id, id of the form the conversation is associated with, conversation name, list of different versions of the conversation and id of the owner of the conversation. • Each version of the conversation is a dictionary with speaker name + timestamp as key with the text as values. Class behavior • Allows retrieving the latest conversation and combining all of the text in the latest version of the conversation into a single string.
ConversationVersion	Class state • Stores information about one version of a conversation, including the version index, timestamp, conversation history, associated extraction run id, source mode, input language, raw transcript, and translated transcript. • The conversation history is stored as a dictionary where each key is usually a speaker name plus timestamp and each value is the spoken text. Class behavior • This class mainly acts as a structured data container. It does not define custom methods, but Pydantic provides validation and serialization behavior for storing and transferring conversation version data.

FormSchema	Class state • Stores information about a form schema including form id, form name, description, owner id, fields, visibility, and collaborators. • The fields are stored as a dictionary where each key is a field name and each value describes the expected type or instruction for that field. Class behavior • This class mainly acts as a structured representation of a form. It does not define custom methods, but it is used by the extraction pipeline to know what information should be extracted from conversations.
ExtractionResult	Class state • Stores the result of an extraction run, including conversation id, form id, filled form data, accepted new fields, run id, and summary. • The filled data is stored as a dictionary containing extracted values for form fields. Class behavior • This class acts as a structured output object for the extraction process. It does not define custom methods, but it standardizes how extraction results are passed between the pipeline, database, and interface layers.
ExtractionRequest	Class state • Stores information needed to extract a single field, including the conversation context, field name, extraction instruction, and original type hint. • The context contains the text from which the field value should be extracted. Class behavior • This class acts as an input object for extraction models. It does not define custom methods, but it groups all information required by the model to answer one extraction request.
RunLog	Class state • Stores information about an extraction run, including run id, conversation id, version index, start time, finish time, extracted fields, status, error message, summary, and owner id. • The status records whether the run is currently running, successful, or failed. Class behavior • This class acts as a structured log for tracking extraction pipeline execution. It does not define custom methods, but it is used to save and retrieve run history, errors, outputs, and summaries.
IConversationRepository	Class state • This class does not store application data directly. It defines an abstract contract for any repository that manages conversation data. Class behavior • Requires implementing methods to retrieve a conversation by conversation id, retrieve all conversations linked to a form id, and save a conversation. • Concrete implementations, such as a MongoDB repository, provide the actual database logic.

IFormRepository	Class state - This class does not store form data directly. It defines an abstract contract for any repository that manages form schemas. Class behavior - Requires implementing methods to retrieve all forms, retrieve a form by form id, save a form, and delete a form by form id. - Concrete implementations decide how and where the form data is stored.
IExtractionModel	Class state - This class does not store extraction results directly. It defines an interface for AI or NLP models that perform information extraction. Class behavior - Defines behavior for processing extraction requests. - It includes a method for extracting values from a batch of ExtractionRequest objects and a method for processing an extraction input string directly. - Concrete implementations may use local Hugging Face models, Ollama, mock models, or a remote model service.
IPipeline	Class state - This class does not store pipeline data directly. It defines an abstract contract for the form-filling pipeline. Class behavior - Requires implementing a run method that takes a conversation id, form id, and version index, then returns an ExtractionResult. - Concrete implementations handle the complete extraction workflow, including loading conversations and forms, running the extraction model, summarizing output, and saving run logs.
IRunLogRepository	Class state - This class does not store run logs directly. It defines an abstract contract for any repository that manages extraction run logs. Class behavior - Requires implementing methods to create a run log, update a run log, retrieve recent run logs, and retrieve a run log by run id. - Concrete implementations provide the actual persistence logic for tracking pipeline execution.
ISummarizer	Class state - This class does not store summary data directly. It defines an abstract contract for summarization models or services. Class behavior - Requires implementing a summarize method that takes text as input and returns a summary string. - Concrete implementations may use local transformer models, Ollama, mock summarizers, or a remote model service.

MongoConversationRepository	Class state • Stores the MongoDB client, selected database, and conversations collection used to persist conversation data. • It is connected using a MongoDB connection string and database name. Class behavior • Provides database operations for conversations. • Allows retrieving a conversation by conversation id, retrieving all conversations associated with a particular form id, and saving a conversation to MongoDB. • When saving, it updates an existing conversation if it already exists, or inserts it if it does not exist.
MongoFormRepository	Class state • Stores the MongoDB client, selected database, and forms collection used to persist form schema data. • It is connected using a MongoDB connection string and database name. Class behavior • Provides database operations for form schemas. • Allows retrieving all forms, retrieving a form by form id, saving a form, and deleting a form by form id. • When saving, it updates an existing form if it already exists, or inserts it if it does not exist.
MongoRunLogRepo	Class state • Stores the MongoDB client, selected database, and run logs collection used to persist extraction run log data. • It is connected using a MongoDB connection string and database name. Class behavior • Provides database operations for extraction run logs. • Allows creating indexes for efficient lookup, creating a new run log, updating an existing run log, retrieving recent run logs, and retrieving a run log by run id. • It is used to track extraction pipeline execution, including status, errors, summaries, and extracted fields.

FormStateModel	Class state • Stores information needed to run the incremental form-state extraction model, including the model path, resolved adapter path, base model id, tokenizer, loaded language model, selected device, maximum number of generated tokens, generation timeout, and background thread executor. • It also stores a tool schema that tells the model to return updates in the form of a structured form state and summary. Class behavior • Provides form extraction using a local Hugging Face/PyTorch causal language model, optionally with a LoRA adapter. • It processes conversation text line by line, keeps track of the current form state, sends each new conversation line to the model, parses the model output, updates the form state, and finally returns extracted answers for the requested fields. • It supports live extraction through <code>process_live_update</code>, where only the latest conversation line is processed using the current field state. • It also supports full extraction through <code>process_extraction_request</code>, where the entire conversation is processed incrementally from the beginning. • The class also provides helper behavior for loading the model, resolving model paths, building prompts, generating model output, parsing tool-call JSON, recovering fields from malformed model output, merging updated form state, and flattening the final state into field-value answers.
QwenSummarizer	Class state • Stores information needed to run a local Qwen summarization model, including the model name, maximum number of generated tokens, selected device, tokenizer, and loaded causal language model. • It also determines whether to run on CUDA or CPU depending on the available hardware. Class behavior • Provides conversation summarization using a local Hugging Face/PyTorch Qwen-Instruct model. • It loads the tokenizer and model during initialization, prepares a chat-style prompt for the input conversation, generates a short summary, and decodes the generated tokens into plain text. • The main public behavior is <code>summarize</code>, which takes conversation text as input and returns a concise summary string. • Internally, it can load the model with GPU support and optional 4-bit quantization when CUDA and BitsAndBytes are available. It runs generation in a background thread so the async application is not blocked while the model is producing the summary.

LocalASRTranscriber	Class state - Stores ASR runtime state including the selected Whisper model name, a lazily initialized Hugging Face ASR pipeline, a thread-pool executor, and device/dtype configuration for CPU or GPU execution. Class behavior - Provides local uploaded-audio transcription using a Whisper-based automatic speech recognition pipeline. - Accepts raw audio bytes plus filename and input language, writes temporary audio files, runs transcription with timestamps, normalizes chunk output, and returns plain transcript text. - Includes a fallback path that reloads audio with librosa if direct pipeline decoding fails. - It is used for static audio upload and ASR preview flows before extraction.
LocalSpeechToText	Class state - Stores STT runtime state including a recognizer instance, a thread-pool executor, an async lock, and the configured model size placeholder for the disabled Whisper path. Class behavior - Provides live speech-to-text transcription for recorded microphone snippets. - Accepts audio bytes plus filename and input language, performs audio normalization, voice activity detection, denoising, and basic inverse text normalization before calling the active SpeechRecognition backend. - Maps language codes to backend-specific language tags and returns cleaned transcript text. - It is used for live mic transcription in the conversation UI and STT-based preview endpoints.
LocalTranslator	Class state - Stores translation runtime state including cached translation models, cached tokenizers, a thread-pool executor, and device selection for CPU or GPU execution. Class behavior - Provides local translation of supported non-English transcripts into English. - Accepts transcript text and source language, lazily loads the matching translation model, runs translation asynchronously, and falls back to returning the original text for English or unsupported languages. - It is used after ASR or STT so downstream extraction can operate on English conversation text.

Container	Class state • Stores shared application dependencies used by the interface layer, including the conversation repository, form repository, run log repository, extraction pipeline, translator, ASR transcriber, and speech-to-text service. • These dependencies are stored as class-level attributes so they can be accessed from different parts of the application after initialization. Class behavior • Provides dependency injection for the application by connecting abstract interfaces to concrete implementations. • The initialize method creates MongoDB repositories, selects the extraction model and summarizer based on configuration settings, creates the form-filling pipeline, and initializes translation and speech services. • It can configure the application to use mock models, a remote model service, Ollama models, or local Hugging Face/PyTorch models. • It acts as the composition root of the system, meaning it is responsible for wiring together the repositories, AI models, summarizer, and pipeline used by the rest of the application.
FormFillingService	Class state • Stores the dependencies required to run the form-filling pipeline, including the conversation repository, form repository, extraction model, run log repository, summarizer, and model processing type. • The model processing type decides whether extraction is done field-by-field or by processing the full conversation state. Class behavior • Provides the main form-filling workflow through the run method. • It creates a run log, retrieves the required conversation and form, prepares the conversation text and form fields, runs the extraction model, runs the summarizer, validates extracted values according to the expected field types, builds the final filled form data, and stores the completed extraction result in the run log. • If the pipeline succeeds, it updates the run log with success status, summary, and extracted fields. • If an error occurs, it updates the run log with failed status and the error message before raising the exception.
AuthService	Class state • Does not store instance state. • Uses module-level session configuration such as the session cookie name, session max age, and signed token serializer. Class behavior • Provides authentication utility operations. • Allows hashing passwords, verifying passwords, creating signed session tokens, reading session tokens, and resolving the current logged-in user from the request cookie. • It is used to manage user login sessions and password validation.

UserRepository	Class state - Does not store instance state. - Uses the MongoDB users collection through the shared application container. Class behavior - Provides database operations for user accounts. - Allows finding users by id, username, email, or multiple ids, inserting users, updating users, deleting users, listing users, and validating username format. - It is used to persist and retrieve authentication and profile data.
AccessPolicy	Class state - Does not store instance state. Class behavior - Provides centralized permission-checking logic. - Allows checking whether a user is an admin, whether an object is global, whether a user can read or write forms, and whether a user can read or write conversations. - It is used to enforce ownership, admin access, global visibility, and collaborative form access rules.
FormQueryService	Class state - Does not store instance state. - Uses the form repository from the shared application container. Class behavior - Provides read-access helpers for form schemas. - Allows retrieving a specific form only if the user has permission to read it, and retrieving all forms visible to a user. - It is used to filter forms based on admin status, ownership, global visibility, and collaborator membership.
ConvoQueryService	Class state - Does not store instance state. - Uses the conversation repository from the shared application container. Class behavior - Provides read-access helpers for conversations. - Allows retrieving a specific conversation only if the user has permission to read it, and listing conversations for a form that are visible to the user. - It is used to prevent users from accessing conversations they do not own unless the conversation is global or the user is an admin.
ConversationParser	Class state - Does not store instance state. Class behavior - Provides conversion operations for conversation text. - Allows parsing raw speaker-labelled conversation text into the internal conversation history format, and rendering stored conversation history back into plain text. - It is used when saving, editing, displaying, and extracting data from conversations.

SchemaBuilder	Class state - Does not store instance state. Class behavior - Provides helper operations for building form schemas. - Allows creating a field schema from parallel field-name and field-question lists, and generating default questions for field names. - It is used when creating or editing forms.
FieldMerger	Class state - Does not store instance state. Class behavior - Provides helper operations for field data normalization and display. - Allows merging extracted fields with accepted new fields, applying reviewed field overrides, normalizing flat field maps, flattening nested field maps, and formatting filled data as HTML. - It is used to prepare extraction results for storage, review, and display.
ExtractionService	Class state - Does not store instance state. - Uses the extraction pipeline model and summarizer from the shared application container. Class behavior - Provides live extraction operations for conversation text. - Allows extracting filled form data, suggested new fields, accepted new fields, and summaries from a conversation. - It is used to run extraction previews, live extraction, ASR extraction, and conversation-based pipeline execution.
OutputRepository	Class state - Does not store instance state. - Uses the MongoDB outputs collection through the shared application container. Class behavior - Provides database operations for saved extraction outputs. - Allows saving extraction results, retrieving an output by run id with access checks, and listing outputs visible to a user. - It is used to persist and retrieve completed extraction results.
TemplateRenderer	Class state - Does not store instance state. - Uses the module-level Jinja2 template environment. Class behavior - Provides a shared template rendering helper. - Allows rendering templates while automatically injecting the current user and admin status into the template context. - It is used by page handlers to keep template responses consistent.

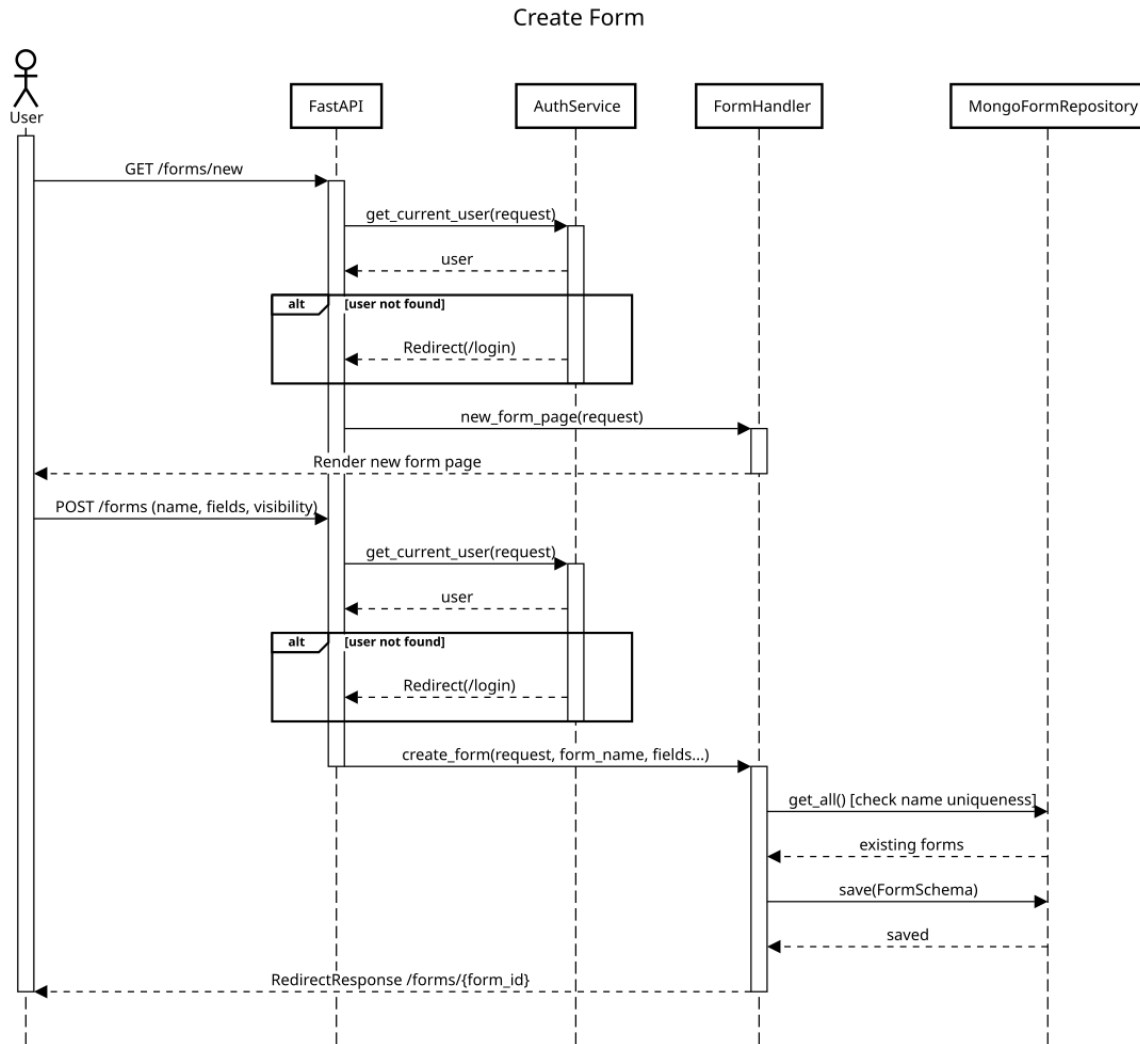
AuthHandler	Class state • Does not store instance state. • Uses authentication services, user repository operations, template rendering, cookies, and redirect responses. Class behavior • Provides request handlers for authentication and profile workflows. • Allows rendering registration and login pages, registering users, logging users in, logging users out, rendering the profile page, changing usernames, and changing passwords. • It is used to handle account creation, login sessions, logout, and user profile updates.
AdminHandler	Class state • Does not store instance state. • Uses authentication, access policy checks, user repository operations, and template rendering. Class behavior • Provides admin-only user management handlers. • Allows listing users, changing a user's role, and deleting user accounts. • It is used by administrators to manage application users while requesting an admin access and unsafe self-deletion or self
FormHandler	Class state • Does not store instance state. • Uses authentication, form repositories, user repositories, schema building, access policy checks, and template rendering. Class behavior • Provides request handlers for form workflows. • Allows rendering the home page, rendering the new form page, creating forms, viewing forms, editing forms, saving form edits, deleting forms, and deleting forms through the API. • It is used to manage form schemas, ownership, visibility, collaborators, and form lifecycle operations.
ConversationHandler	Class state • Does not store instance state. • Uses authentication, form query services, conversation query services, conversation parsing, persistence helpers, and template rendering. Class behavior • Provides request handlers for conversation workflows. • Allows rendering conversation entry pages, creating conversations, optionally extracting immediately after creation, listing conversations for a form, viewing conversations, editing conversations, saving new conversation versions, and entering collaborative conversation rooms. • It is used to manage conversation input, storage, versioning, and navigation around form-linked conversations.

ExtractionHandler	Class state - Does not store instance state. - Uses authentication, form and conversation query services, the extraction pipeline, output storage, field merging, and template rendering. Class behavior - Provides request handlers for extraction workflows. - Allows running extraction for a saved conversation, previewing extraction from raw conversation text, serving live extraction API requests, and rendering the live extraction page. - It is used to generate filled form data, summaries, accepted new fields, and extraction result displays.
ASRHandler	Class state - Does not store instance state. - Uses authentication, form query services, audio upload data, ASR transcription services, diarization services, translation services, conversation persistence, and extraction helpers. Class behavior - Provides request handlers for speech-to-text and audio-based extraction workflows. - Allows rendering the ASR extraction page, creating conversations from uploaded or recorded audio, translating transcripts to English, handling diarized speaker turns, transcribing live audio, and previewing translated ASR output. - It is used to convert speech input into conversation text and run extraction from audio-based conversations.
OutputHandler	Class state - Does not store instance state. - Uses authentication, output repository operations, run log repository operations, conversation query services, field merging, access policy checks, and template rendering. Class behavior - Provides request handlers for saved outputs and pipeline run logs. - Allows viewing saved extraction outputs, viewing output details by run id, viewing recent run logs, and viewing a specific run log. - It is used to inspect completed extraction results and administrator-only pipeline execution logs.

Sequence Diagram(s)

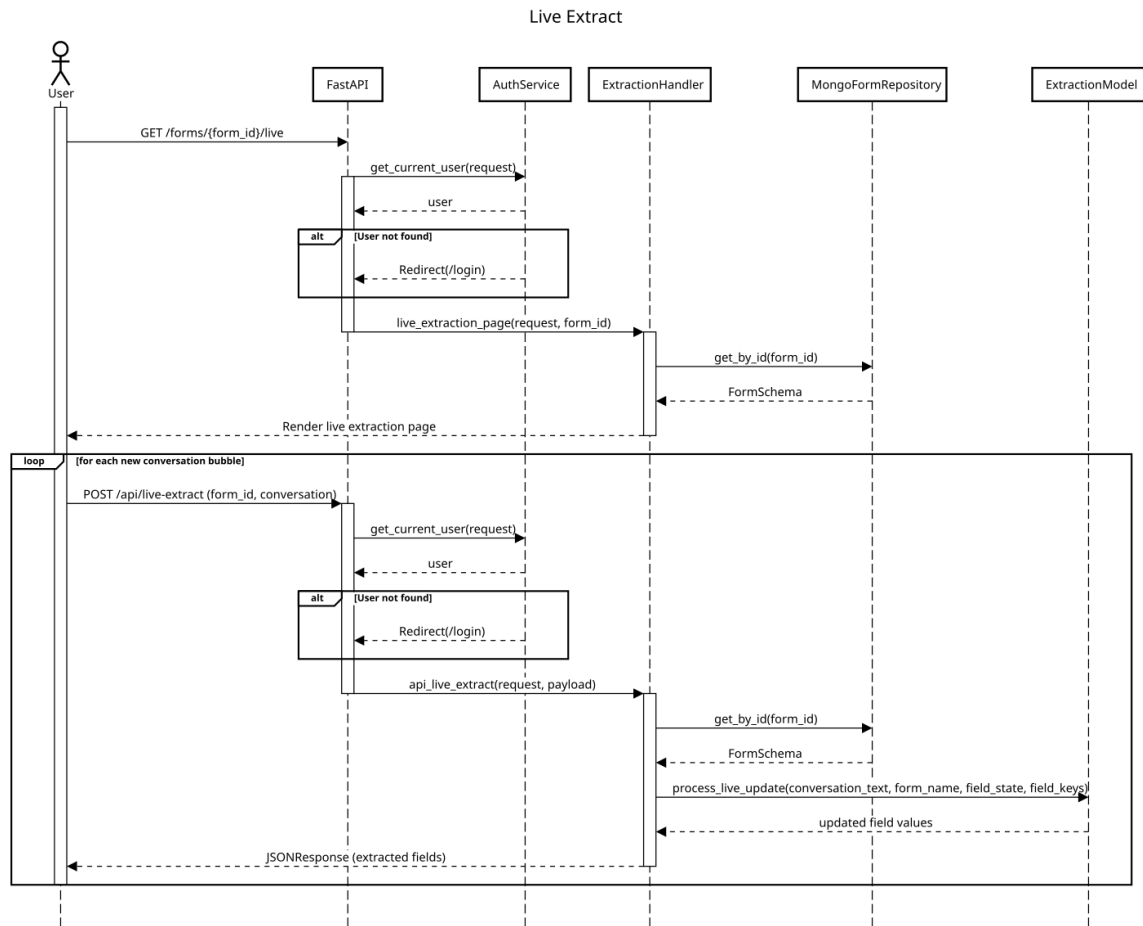
1. Create form

Every user in the system can create a form for personal use. Only admins can create global forms that are accessible to all users in the system, any edits to which must be saved as a new form. It is mandatory that a form has a name and at least one field. Form description may help identifying the form later and is a recommended operation. Consider the sequence diagram for creating a new form for personal use as below:



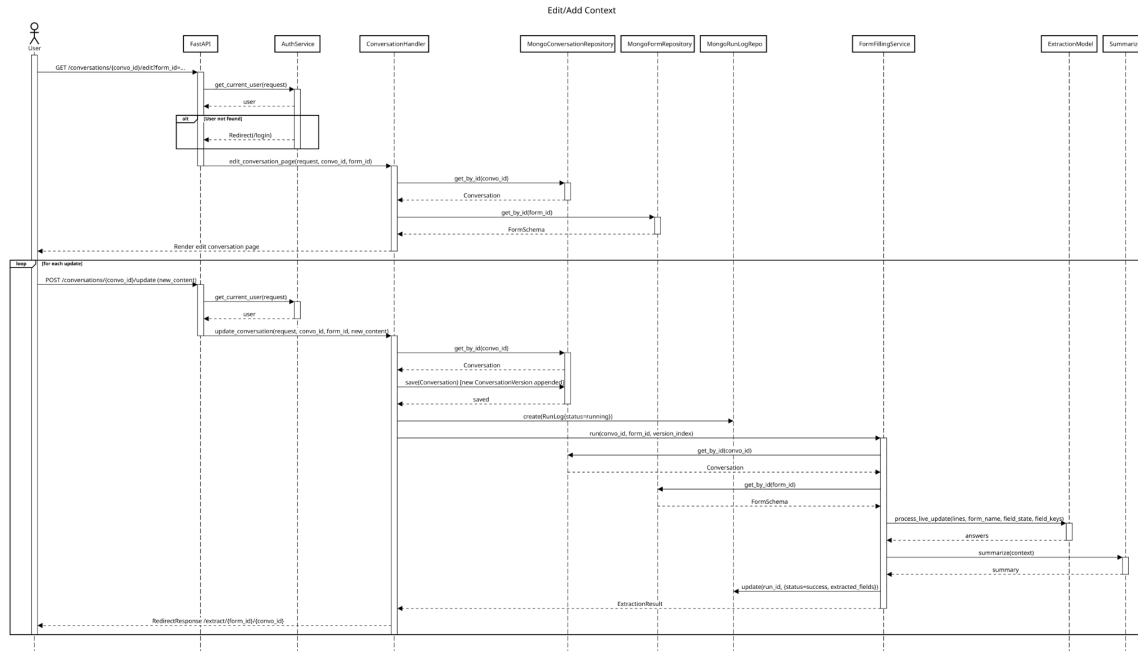
2. Live Extraction - from Textual input

The User is presented with a chat-like interface with toggle-able speaker bubbles. The User can type out a conversation and the pipeline is run after each keydown operation i.e. after every bubble enters. The User can also edit fields to their liking at any time during the extraction and finally at the end. This ensures a certain level of reliability. Consider the sequence diagram of Live Extraction as below:



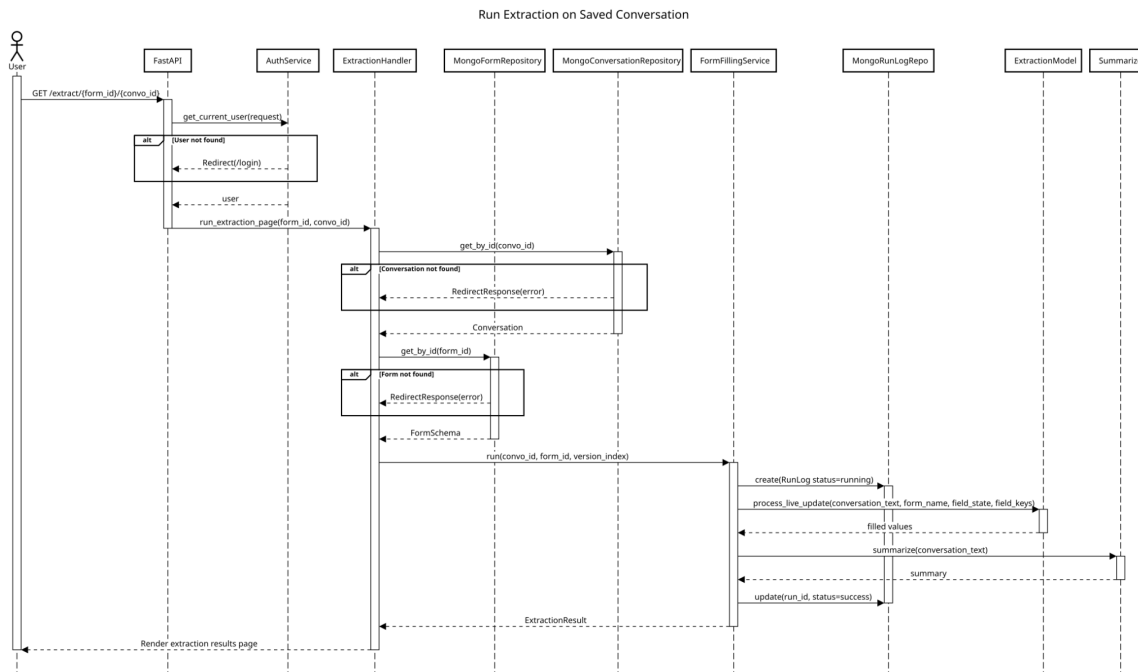
3. Edit an existing conversation

A user may edit an existing conversation and observe live extraction simultaneously on the form. Editing operations include adding, removing, changing speaker types and adding, removing or changing message bubbles and their content. The live extraction pipeline is triggered after every edit as described before. Consider the sequence diagram for the same below:



4. Run Extraction on saved conversation

A User may decide to run an extraction on a saved conversation. In this case, the entire conversation is sent to the pipeline and the filled values are rendered in one-shot instead of the iterative method used for live extracts. Consider the sequence diagram as below:



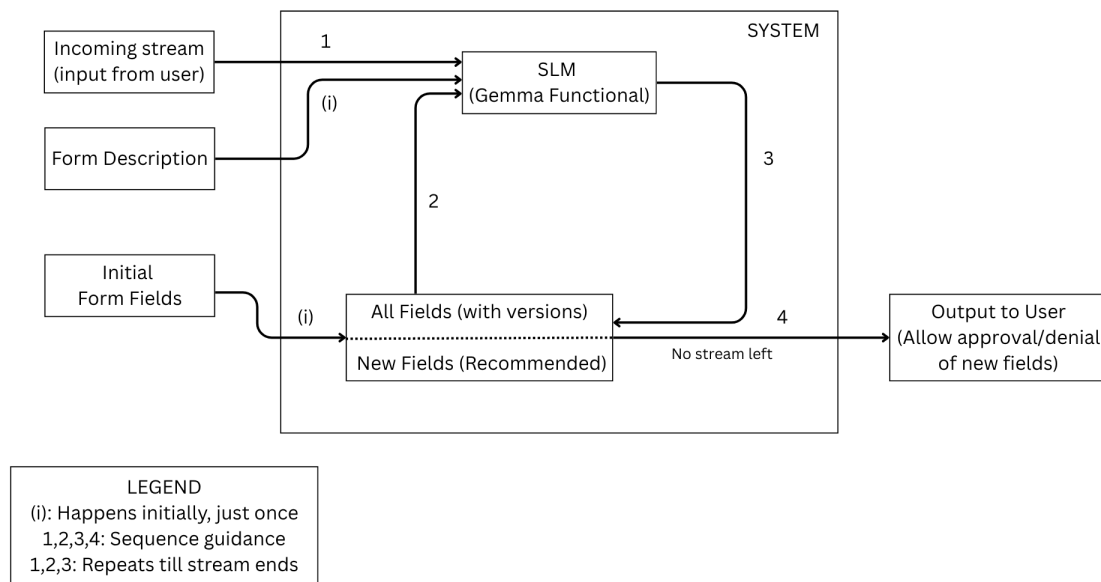
5. Run Extraction on an audio conversation

Consider the complex sequence diagram below that explains the flow of control for an ASR usecase:

The project was distributed over 4 broad phases throughout the semester. For phase 0, the team had to prepare a CLI version of the pipelined workflow for a static text conversation and present the same to the client with stubs for the model.

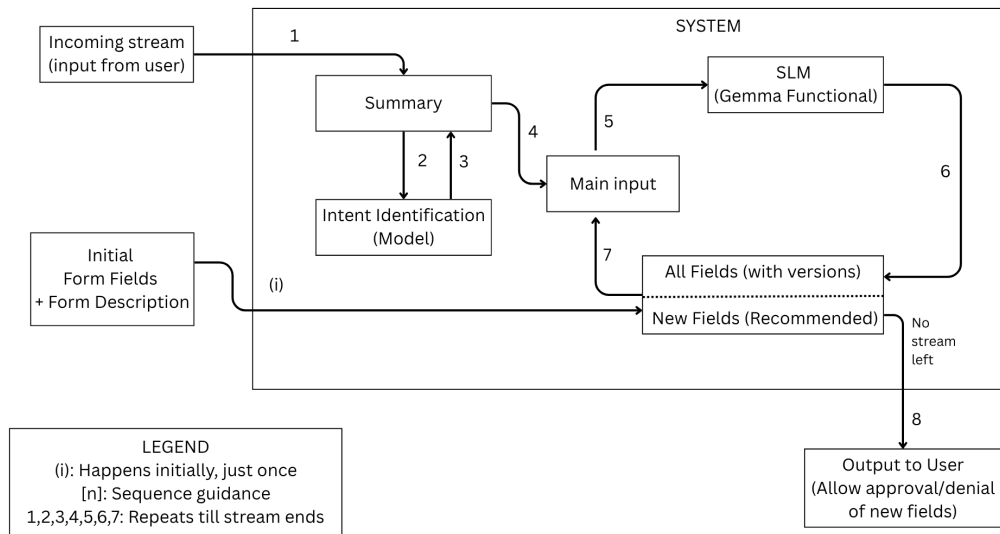
For phase 1, the CLI version was replaced with a simple wireframe along with a simple QA model DistilBert. No architectural designs were considered for this phase and the model was able to accurately fill forms for heavily-guided conversations. Additionally, during this phase, we restructured our codebase to follow Domain Driven Design (DDD) principles. This was done to ensure the code is very modular and different sections of the codebase can be worked on by different developers without any conflicting updates.

For phase 2, the following architecture diagram depicts the design of the AI workflow with the Gemma Functional being the SLM of choice. The form description and initial fields are seeded once, and the state (which consists of all fields along with new ones, if applicable) are iteratively updated with the incoming stream of text as provided by the user. The model performance exceeded that from the previous phase. However, there was still room for improvement.



For phase 3, as per client suggestions a separate intent identification model was injected into the previous architectural diagram. The main motive was to reduce the length of input to the main model by summarizing the intent of the previous X conversation bubbles processed.

Due to lack of an appropriate model to achieve the same, the main model posed itself as the summarizer thus aiding the process flow as needed. No obvious improvement with this architecture was noticed.



FINAL ARCHITECTURE:

The final architecture diagram had multiple updates over the previous version. Firstly, we moved from fine tuning Gemma Functional models to fine tuning Qwen models instead. Finetuned Qwen-3-4B is the model which gave the most accurate results. Additionally, the current summary was not passed to the extraction model in each iteration. The Summarizer and the Form Field Extraction models are run as independent pipelines.

